



✓ Technical Assignment – Vehicle Management System for a Car Rental Company

Background

DriveNow is a car rental company that manages a fleet of vehicles.

The company wants to develop an internal system that will allow:

- Managing vehicles (add, update, delete)
- Registering rentals
- Displaying each vehicle's status (available / in use / under maintenance)

The system should be designed as a foundation for future expansion — therefore, it's important to maintain clean architecture, good engineering practices, and clear separation of concerns.

✓ Objectives

Your goal is to develop a **Python-based service** (either a REST API or a simple CLI tool) that manages this data. The system should include:

0. Design of your System architecture
 1. A data access layer (database)
 2. Business logic layer
 3. User interface (API or CLI)
 4. Logging and metrics
 5. Basic documentation (README)
-

Technical Requirements

1. Database

- Use any relational database: **MySQL / MongoDB / Other**. Explain your choice.
- Define a basic schema with the following tables:
 - **cars** – car ID, model, year, status
 - **rentals** – rental ID, car ID, customer name, start date, end date
- Implement data access via an ORM.

2. Required Operations

- Add a new car

- Update car details (e.g., change status)
- List all cars (with optional status filter)
- Register a new rental
- End a rental and update car status accordingly

3. Logging

- Use Python's built-in logging module
- Log critical actions (e.g., add/update/error/end rental)
- Support logging both to console and to a file

4. Metrics

- Collect basic metrics (using `prometheus_client` or another library), such as:
 - Number of active cars
 - Number of ongoing rentals
 - Average request/operation response time

5. Architecture and Code Quality

- Follow good engineering practices:
 - **Separation of layers** (data access / services / API)
 - **SOLID principles** where applicable
 - Clean, readable, and well-documented code
- Include at least **4 unit tests**

Message Queue Communication (EXTRA, optional, recommended)

- The system should be designed with a clear separation that allows for easy maintenance of code and components.
- Communication should be implemented using a message queue infrastructure of your choice.

6. Environment Setup

- The project should run as a standalone Python application
- Include a requirements file
- Add a `docker-compose.yml` for setup

7. GIT

- Host the solution in a **public GitHub or GitLab repository**
- Use clear and descriptive commit messages
- Prefer working in a dedicated **feature branch**

☑ Deliverables

1. Full source code
2. A `README.md` file that explains:
 - Graphic flow of your design architecture
 - How to run the project
 - How to use the API or CLI
 - A brief architecture description
 - Example usage
 - Screenshots (Recommended)
3. A link to the Git repository

Good Luck!